

The Challenge: Maximum Sum of Distinct Subarrays With Length K

Problem Statement

Given an integer array `nums` and an integer `k`, find the maximum sum of all subarrays of `nums` that meet the following conditions:

1. The length of the subarray is exactly `k`.
2. All the elements of the subarray are **distinct** (no duplicate numbers).

Return the maximum subarray sum. If no subarray meets the conditions, return 0.

Constraints (Standard Banking Test Scale)

- $1 \leq k \leq \text{nums.length} \leq 10^5$
- $1 \leq \text{nums}[i] \leq 10^5$
- **Time Limit:** 1.0 second (Requires an $O(N)$ solution. A brute-force $O(N \times k)$ solution will trigger a **Time Limit Exceeded (TLE)** error).

Examples

Example 1:

- **Input:** `nums = [1, 5, 4, 2, 9, 9, 9], k = 3`
- **Output:** 15
- **Explanation:** The subarrays of length 3 are:
 - `[1, 5, 4]`: All distinct, sum = 10
 - `[5, 4, 2]`: All distinct, sum = 11
 - `[4, 2, 9]`: All distinct, sum = 15
 - `[2, 9, 9]`: Contains duplicates (9). Invalid.
 - `[9, 9, 9]`: Contains duplicates (9). Invalid.
 - The maximum valid sum is **15**.

1. Standard / Happy Path Cases

These test typical expected inputs where multiple valid windows exist.

- **Case 1: Standard mix with a clear winner**
 - **Input:** `nums = [1, 5, 4, 2, 9, 9, 9], k = 3`
 - **Expected Output:** 15
 - **Explanation:** The valid windows of size 3 are `[1, 5, 4]` (sum 10), `[5, 4, 2]` (sum 11), and `[4, 2, 9]` (sum 15). The windows containing multiple 9s are invalid. Maximum sum is 15.
- **Case 2: All elements unique**
 - **Input:** `nums = [1, 2, 3, 4, 5], k = 2`
 - **Expected Output:** 9
 - **Explanation:** All adjacent pairs are distinct. The last window `[4, 5]` gives the maximum sum of 9.

2. Edge Cases (Duplicates & Invalidation)

These test whether your algorithm correctly identifies and handles duplicate elements within the window.

- **Case 3: No valid subarray due to global duplicates**
 - **Input:** `nums = [4, 4, 4, 4], k = 2`
 - **Expected Output:** 0
 - **Explanation:** Every possible subarray of length 2 contains duplicate 4s. No valid window exists.
- **Case 4: Trapped duplicate in the middle**
 - **Input:** `nums = [1, 2, 1, 3, 4], k = 3`
 - **Expected Output:** 8
 - **Explanation:** `[1, 2, 1]` is invalid. `[2, 1, 3]` is valid (sum 6). `[1, 3, 4]` is valid (sum 8). Max sum is 8.
- **Case 5: Duplicates separated exactly by distance k**
 - **Input:** `nums = [1, 2, 3, 1, 5], k = 3`
 - **Expected Output:** 9
 - **Explanation:** `[1, 2, 3]` is valid (sum 6). `[2, 3, 1]` is valid (sum 6). `[3, 1, 5]` is valid (sum 9). Note that when the window moves to `[2, 3, 1]`, the first 1 has left the window, so it becomes valid again.

3. Boundary Cases (Constraints & Sizes)

These test the extreme boundaries of the input constraints.

- **Case 6: Window size k equals array length**
 - **Input:** `nums = [1, 2, 3, 4], k = 4`
 - **Expected Output:** 10
 - **Explanation:** The entire array is the only window, and all elements are distinct.
- **Case 7: Window size k equals array length with duplicates**
 - **Input:** `nums = [1, 2, 3, 2], k = 4`

- **Expected Output:** 0
 - **Explanation:** The only window of size 4 contains duplicate 2s, making it invalid.
- **Case 8: Minimal constraints (\$k = 1\$)**
 - **Input:** `nums = [5, 1, 9, 2]`, `k = 1`
 - **Expected Output:** 9
 - **Explanation:** A window of size 1 can never have duplicates. The answer is simply the maximum single element.